The Session Layer

Igor Oliveira Souza e Silva - 12311BCC017;
Joao Pedro Costa Baroni - 12311BCC035;
Leonado Henrique Carvalho Oliveira - 12311BCC003;
Paulo David Gonçalves Souza -

Chp. 07 - The Session Layer

7. Intro

7.1- Session Layer Design Issues

7.1.1- Services Provided to the Presentation Layer

7.1.2 - Data Exchange

7.1.3 - Dialog Management

7.1.4 - Synchronization

7.1.5 - Activity Management

Chp. 07 - The Session Layer

7.1.6 - Exception Reporting

7.1.7 - The OSI Session Service Primitives

7.2 - Remote Procedure Call

7.2.1 - The Client-Server Model

7.2.2 - Implementation of Remote Procedure Call

7.2.3 - Implementation of Remote Procedure Call

7.2.4 - Orphans

Chp. 07 - The Session Layer

7.2.5 - Discussion of RPC

7.3 - Examples of the Session Layer

7.3.1 - The Session Layer in Public Networks

7.3.2 - The Session Layer in the ARPANET

7- Intro

- As **camadas de sessão, apresentação e aplicação** formam as camadas superiores do Modelo de Referência OSI.

- ... Elas pegam o canal básico e livre de erros fornecido pela **camada de transporte** e adicionam recursos adicionais que são úteis para uma ampla variedade de aplicações, para que os desenvolvedores dessas aplicações não tenham que **reimplementar esses recursos repetidamente** como parte de cada programa separado.

- Antes do OSI, nenhuma arquitetura de rede possuía uma **camada de sessão dedicada**. Serviços semelhantes existiam em outras arquiteturas (como o SNA da IBM), mas estavam dispersos por diferentes camadas.

7- Intro

- ... A inclusão desta camada foi alvo de debate considerável. A proposta britânica, por exemplo, defendia um modelo com **apenas cinco camadas, excluindo a de sessão.**

- ...“ **Thin Layer**”...: Apesar de incluída, a camada de sessão é reconhecidamente uma camada com poucos recursos ("thin") se comparada às camadas inferiores (como rede e transporte).

- ... A camada de sessão possui recursos opcionais e de **importância relativa**, pois muitos de seus serviços podem ser desativados, tornando-a simplificada, e é considerada menos crucial que outras camadas (como a de transporte), sendo dispensável para muitas aplicações.

7.1- SESSION LAYER DESIGN ISSUES

- ... Nesta apresentação, discutiremos algumas das questões de projeto **relevantes para a camada de sessão.**

- ... Estas incluem, entre outras, **o gerenciamento de diálogo, a sincronização e o gerenciamento de atividades.**

- ... Todos eles podem ser considerados serviços de **valor agregado** colocados sobre a conexão de transporte básica.

7.1.1 - SERVICES PROVIDED TO THE PRESENTATION LAYER

- ... A **principal função** da camada de sessão é fornecer uma maneira para os usuários de sessão (por exemplo, entidades de apresentação ou, às vezes, apenas processos de usuário comuns) **estabelecer conexões**, chamadas de sessões, e **transferir dados de forma ordenada por meio delas.**

- ... Embora primitivas sem conexão estejam disponíveis na camada de sessão, uma **sessão sem conexão não pode fazer uso dos recursos orientados ao usuário** para os quais a camada de sessão foi projetada.

7.1.1 - SERVICES PROVIDED TO THE PRESENTATION LAYER

- ... A camada de sessão fornece serviços à **camada de apresentação**

- ... Sua posição na hierarquia é mostrada na Fig. 7-1.

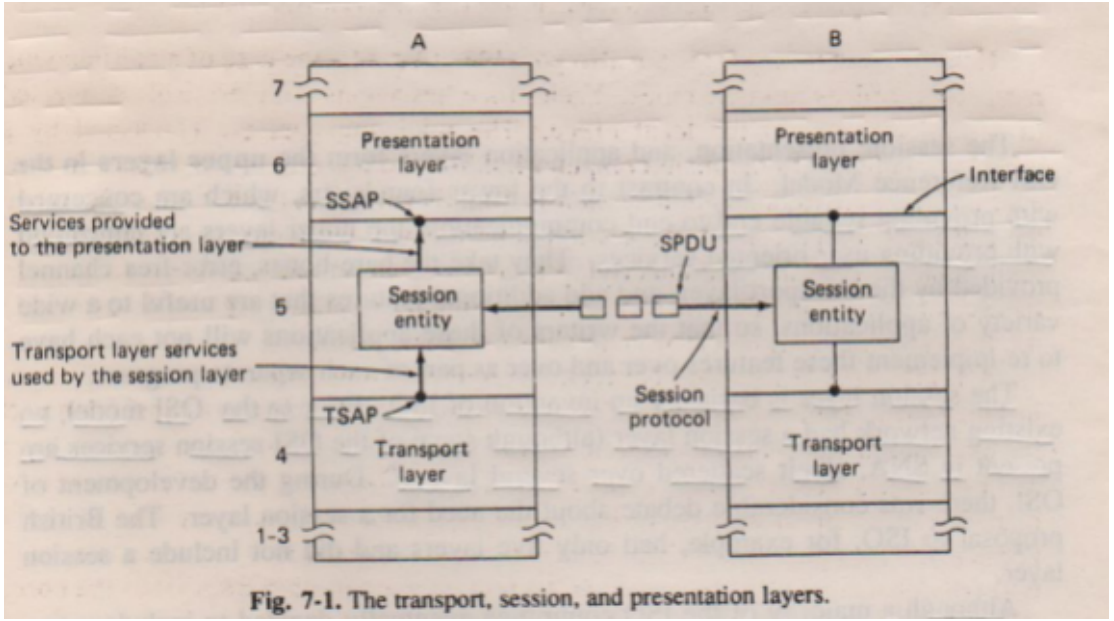


Fig. 7-1. The transport, session, and presentation layers.

7.1.1 - SERVICES PROVIDED TO THE PRESENTATION LAYER

- ... A camada de sessão fornece serviços à **camada de apresentação**

- ... Aqui, os pontos de acesso ao serviço são chamados de pontos de acesso ao serviço de sessão **(SSAPs**, na sigla em inglês).

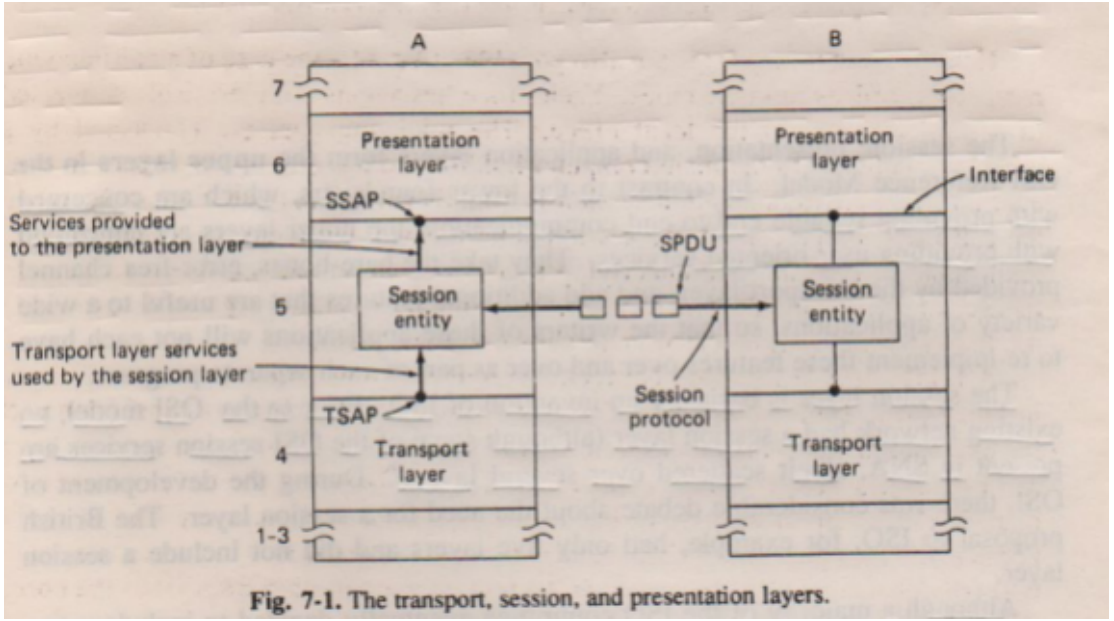


Fig. 7-1. The transport, session, and presentation layers.

7.1.1 - SERVICES PROVIDED TO THE PRESENTATION LAYER

- ... A camada de sessão fornece serviços à **camada de apresentação**

- ... As unidades de dados de protocolo são chamadas de unidades de dados de protocolo de sessão (**SPDUs**, na sigla em inglês).

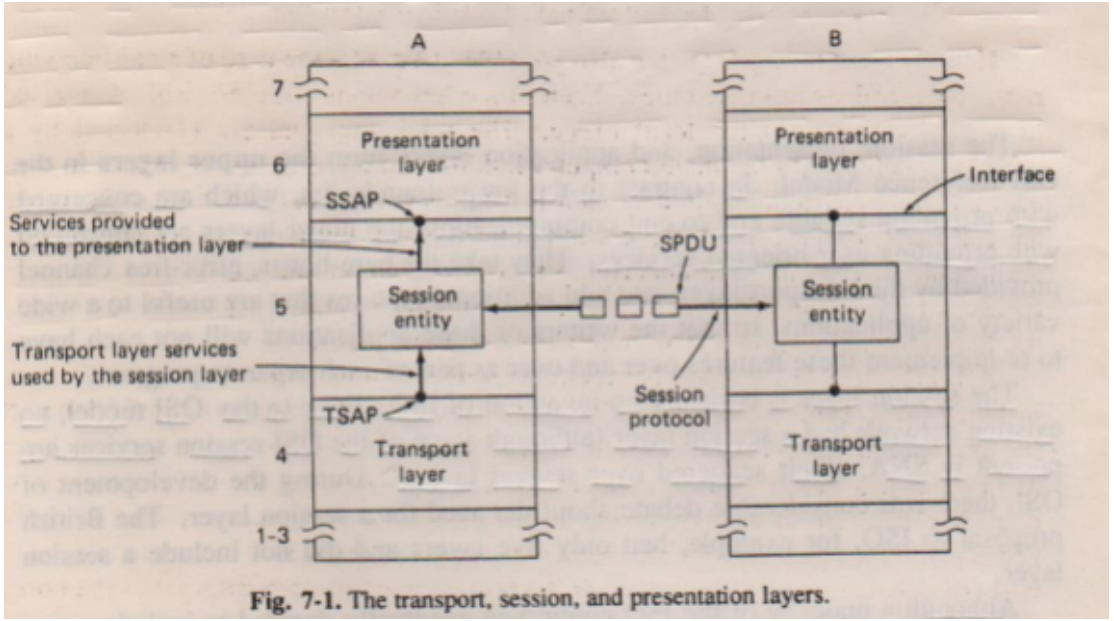


Fig. 7-1. The transport, session, and presentation layers.

7.1.1 - SERVICES PROVIDED TO THE PRESENTATION LAYER

- ... Uma sessão tem uma estreita semelhança com uma **conexão de transporte,** mas as duas não são idênticas. Geralmente, quando uma **solicitação chega à camada de sessão** para estabelecer uma sessão, uma **conexão de transporte deve ser estabelecida** para transportar a conexão.

- ... Quando a sessão é **encerrada,** a conexão de **transporte é liberada**. Neste exemplo, há um mapeamento um-para-um entre a sessão e a conexão de transporte. Esta situação é ilustrada na Fig. 7-2(a).

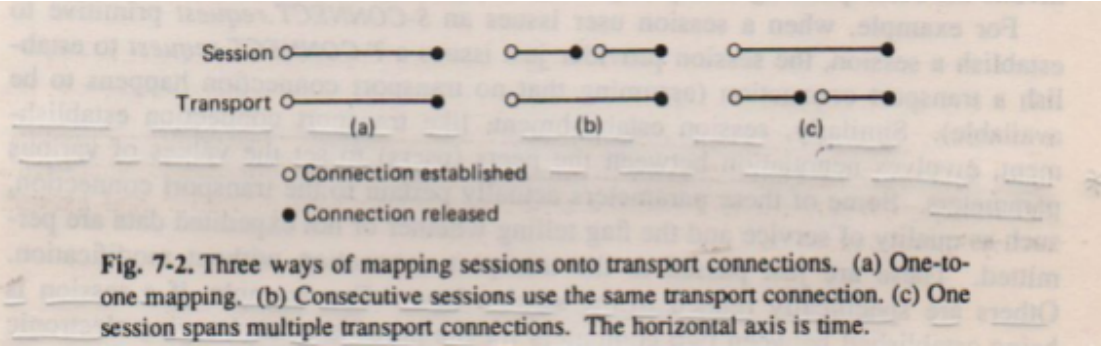


Fig. 7-2. Three ways of mapping sessions onto transport connections. (a) One-to-one mapping. (b) Consecutive sessions use the same transport connection. (c) One session spans multiple transport connections. The horizontal axis is time.

7.1.1 - SERVICES PROVIDED TO THE PRESENTATION LAYER

- ... **No entanto**, outros mapeamentos também são possíveis.
- ... Considere o caso a seguir: “Sistema de reservas de uma companhia aérea com agentes em escritórios locais conectados a um computador central, em que, **estabelecer e encerrar uma conexão de transporte para cada sessão** (atendimento de chamada) é operacionalmente custoso.”
- ... Dessa maneira, é mais simples permitir que **sessões sequenciais usem a mesma conexão de transporte**, como mostra a Fig. 7-2(b)

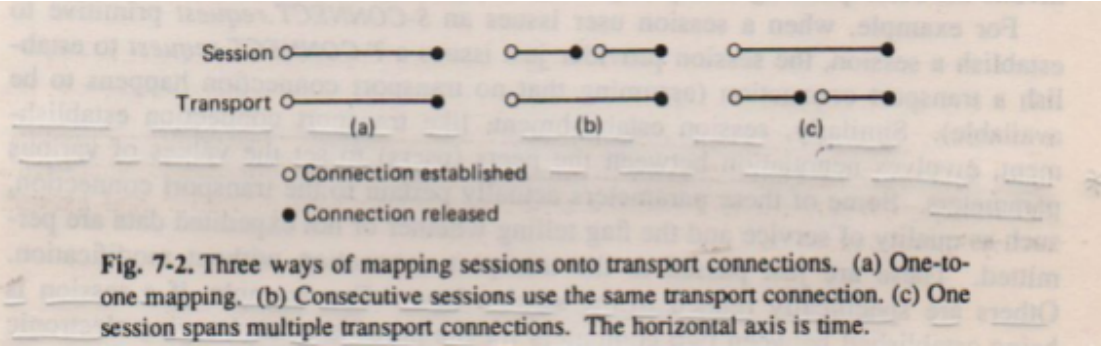


Fig. 7-2. Three ways of mapping sessions onto transport connections. (a) One-to-one mapping. (b) Consecutive sessions use the same transport connection. (c) One session spans multiple transport connections. The horizontal axis is time.

7.1.1 - SERVICES PROVIDED TO THE PRESENTATION LAYER

- ... A Fig. 7-2(c) ilustra um terceiro mapeamento possível, no qual **uma única sessão de longa duração** pode abranger **múltiplas conexões de transporte** sequenciais.
- ... Esse modelo é projetado para **oferecer resiliência**: se uma conexão de **transporte falhar**, a camada de sessão é capaz de **estabelecer uma nova conexão e continuar a sessão** de forma transparente sobre ela
- ... Esta abordagem é crucial quando as **entidades de transporte estão externas aos hosts**, pois a camada de sessão assume o papel de camada de software mais baixa com capacidade de sobreviver a falhas da sub-rede, tornando-se responsável pela recuperação de falhas externas.

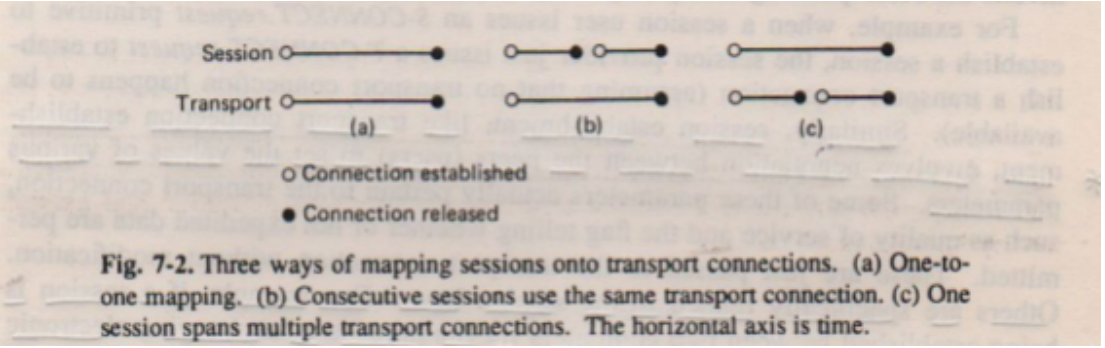


Fig. 7-2. Three ways of mapping sessions onto transport connections. (a) One-to-one mapping. (b) Consecutive sessions use the same transport connection. (c) One session spans multiple transport connections. The horizontal axis is time.

7.1. 2 - DATA EXCHANGE

- ... A característica mais importante da camada de sessão é a **troca de dados**. Uma sessão, assim como uma conexão de transporte, passa por três fases: **estabelecimento, uso e liberação.**
- ... As **primitivas fornecidas à camada de apresentação** para estabelecer, usar e liberar sessões são muito **semelhantes às primitivas fornecidas à camada de sessão** para estabelecer, usar e liberar conexões de transporte.
- ... Em muitos casos, **tudo que a entidade de sessão precisa fazer** quando uma primitiva é invocada pelo usuário da sessão **é invocar a primitiva de transporte correspondente** para realizar o trabalho.
- ... Por exemplo, quando um usuário de sessão emite uma primitiva **`S-CONNECT.request`** para estabelecer uma sessão, o provedor de sessão simplesmente emite um **`T-CONNECT.request`** para estabelecer uma conexão de transporte (supondo que nenhuma conexão de transporte esteja disponível no momento).
- ... Da mesma forma, **o estabelecimento de sessão**, assim como o estabelecimento de conexão de transporte, **envolve negociação entre os pares** (usuários) para definir os valores de vários parâmetros.

7.1. 2 - DATA EXCHANGE

- ... Estes são simplesmente passados para a conexão de transporte **sem modificação**. Outros estão **especificamente relacionados** à camada de sessão.

- ... Por exemplo, **se uma sessão está sendo estabelecida** entre dois computadores para trocar correio eletrônico em ambas as direções, **um dos parâmetros da sessão pode especificar qual lado deve iniciar**.
- ... No entanto, apesar dessas semelhanças, **existem diferenças importantes entre uma sessão e uma conexão de transporte.** A principal delas é a forma como as **sessões e as conexões de transporte são liberadas.**
- ... As conexões de transporte são terminadas com a primitiva `**T-DISCONNECT.request**`, que realiza uma liberação abrupta e pode resultar na perda irremediável de dados em trânsito no momento da liberação

- ...As sessões são encerradas com a primitiva `**S-RELEASE.request**`, que resulta em uma liberação ordenada (também chamada de liberação graciosa), na qual nenhum dado é perdido. Uma liberação abrupta, como `**T-DISCONNECT.request**`, também está disponível na forma de `**S-U-ABORT.request**`

7.1. 2 - DATA EXCHANGE

- ... A diferença entre uma liberação ordenada e uma abrupta pode ser vista na **Fig. 7-3.**
- ... **Na liberação abrupta**, assim que um usuário emite a primitiva apropriada para desconectar, nenhum dado adicional pode ser entregue a ele.
- ... Na **Fig. 7-3(a)**, uma vez que B invocou `**T-DISCONNECT.request**`, sua extremidade da conexão é imediatamente liberada. Ele pode não receber a mensagem que está atualmente em trânsito para ele, mesmo que essa mensagem chegue antes que a **TPDU DR (Disconnect Request)** chegue a A.

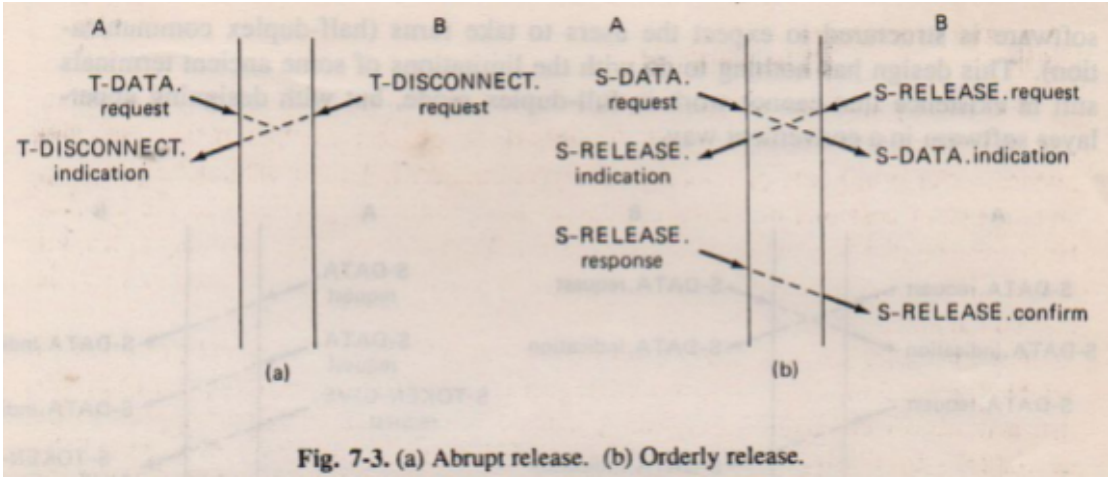


Fig. 7-3. (a) Abrupt release. (b) Orderly release.

7.1. 2 - DATA EXCHANGE

- ... A diferença entre uma liberação ordenada e uma abrupta pode ser vista na **Fig. 7-3.**
- ... **Na liberação abrupta**, assim que um usuário emite a primitiva apropriada para desconectar, nenhum dado adicional pode ser entregue a ele.
- ... Além disso, A pode não recusar a solicitação para liberar a conexão. O provedor de serviço de transporte simplesmente emite uma indicação `T-**DISCONNECT.indication**` e pronto. É como uma conexão telefônica: **quando uma das partes desliga, a conexão é terminada, não importa o que a outra queira.**

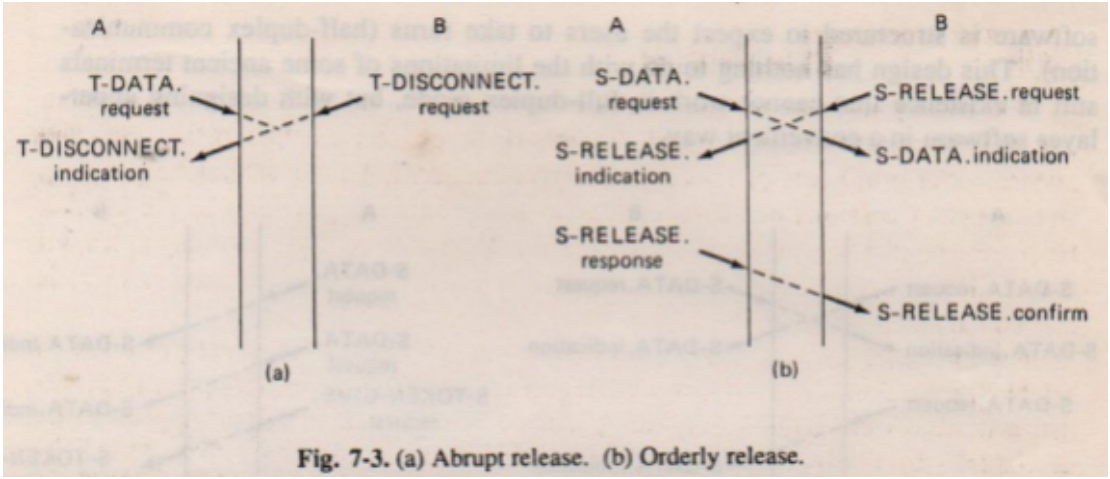


Fig. 7-3. (a) Abrupt release. (b) Orderly release.

7.1. 2 - DATA EXCHANGE

- ... A diferença entre uma liberação ordenada e uma abrupta pode ser vista na **Fig. 7-3.**
- ... A liberação ordenada funciona de maneira diferente. Ela usa um **handshake completo,** com primitivas de **request** (requisição), **indication** (indicação), **response** (resposta) e **confirm** (confirmação).
- ...Na Fig. 7-3(b), mesmo depois que B emitiu uma primitiva `**S-RELEASE.request**`, ele ainda pode aceitar mensagens até que A tenha confirmado a liberação. A simples emissão de um `**S-RELEASE.request**` não encerra a sessão por si só.

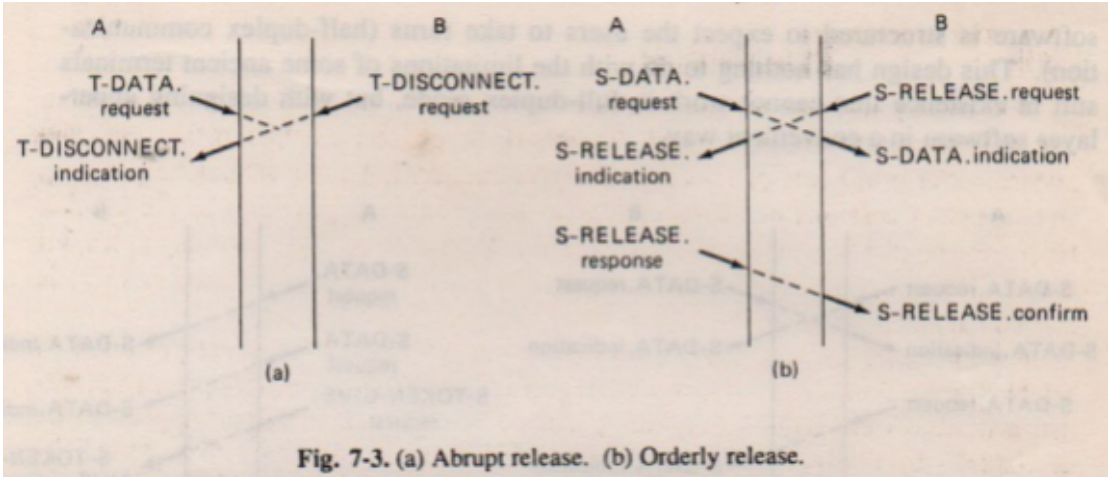


Fig. 7-3. (a) Abrupt release. (b) Orderly release.

- ... O usuário remoto deve concordar. Se o usuário remoto desejar continuar a sessão, ele pode rejeitar a tentativa de liberação (definindo um parâmetro na primitiva `**S-RELEASE.response**`), e a sessão continua como se nada tivesse acontecido. **Uma sessão só é terminada quando ambas as partes tiverem tido o suficiente.**

7.1. 2 - DATA EXCHANGE

- ... Durante a elaboração do padrão da camada de transporte para o governo dos Estados Unidos, **o National Bureau of Standards (NBS)** considerou essa questão suficientemente relevante e optou por incorporar uma opção de liberação ordenada à camada de transporte.
- ... Essa decisão resolveu o problema em questão, mas **simultaneamente resultou em um padrão americano distinto** do adotado pelo resto do mundo.

- ... Profissionais envolvidos na criação de padrões frequentemente enfrentam o dilema entre seguir a abordagem tecnicamente mais adequada ou alinhar-se às práticas já consolidadas globalmente. **Essa tensão constante contribui para a proliferação de múltiplos padrões.**

7.1. 2 - DATA EXCHANGE

- ... O endereçamento é outra área em que as **camadas de sessão e transporte diferem**, embora apenas ligeiramente.

- ... Para estabelecer uma sessão, deve-se especificar o **endereço SSAP (em vez do endereço TSAP**) ao qual se conectar.

- ... Embora os padrões não digam como os endereços SSAP devem ser construídos, é provável que, na prática, um **endereço SSAP consista em um endereço TSAP mais algumas informações identificadoras adicionais.**

7.1. 2 - DATA EXCHANGE

- ... Mais uma maneira pela qual a troca de dados de sessão difere da troca de dados de transporte é o **número de tipos diferentes de dados.**

- ... A camada de transporte possui **dois fluxos de dados logicamente independentes:** dados regulares e dados acelerados. A camada de sessão **possui ambos esses fluxos e mais dois** (dados tipados e dados de capacidade).

- ... Os outros dois fluxos estão relacionados a características da camada de sessão que ainda não abordamos, portanto, **adiaremos nossa discussão sobre eles para mais adiante neste capítulo.**

7.1. 3 - DIALOG MANAGEMENT

- ... Em princípio, **todas as conexões OSI são full duplex**, ou seja, as PDUs podem se mover em ambas as direções pela mesma conexão simultaneamente. A comunicação full duplex é mostrada na Fig. 7-4(a).

- ... No entanto, existem muitas situações em que o software de camada superior é **estruturado para esperar que os usuários se alternem (comunicação half duplex).**

- ... Este design não tem relação com as limitações de alguns terminais antigos ainda existentes que não podem operar em modo full duplex, mas sim com o **design conveniente do software de camada superior.**

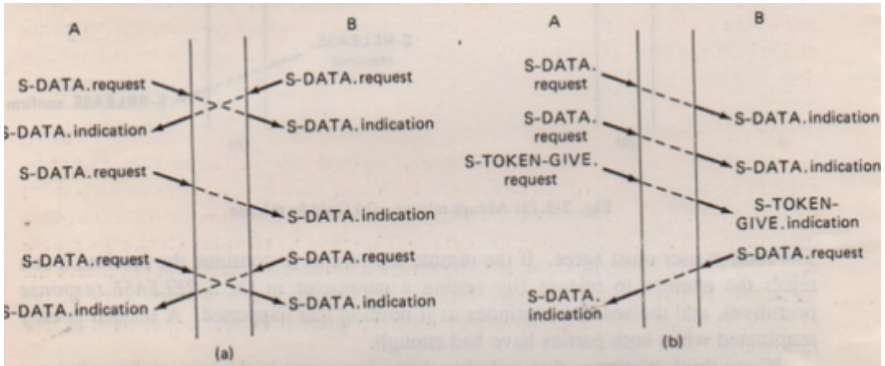


Fig. 7-4. (a) Full-duplex communication. (b) Half-duplex communication managed with a token.

7.1. 3 - DIALOG MANAGEMENT

- ...Como exemplo, **considere um sistema de gerenciamento de banco de dados que pode ser acessado a partir de terminais remotos** (por exemplo, reservas de companhias aéreas ou home banking)

- ...O modo de operação mais natural é o usuário **enviar uma consulta ao sistema de banco de dados e então aguardar a resposta.**

- ...Permitir que os usuários enviem uma segunda e terceira consulta antes que a primeira seja respondida **complica desnecessariamente o sistema.**

- ... Logicamente, **é desejável operar o sistema em modo half duplex**: ou é a vez do usuário transmitir ou do sistema de banco de dados.

- ...Controlar de quem é a vez de falar (e impor isso) é chamado de **gerenciamento de diálogo** e é um dos serviços que podem ser **fornecidos pela camada de sessão quando solicitado**.

7.1. 3 - DIALOG MANAGEMENT

- ...A maneira como o gerenciamento de diálogo é implementado é através do uso de um **token de dados**
- ...Quando uma sessão é estabelecida, **a operação half duplex é uma das opções que pode ser selecionada**. Se a operação half duplex for escolhida, **a negociação inicial também determina qual lado obtém o token primeiro**.
- ...Somente o usuário que detém o token pode transmitir dados; o outro deve permanecer em silêncio. Quando o **detentor do token termina de transmitir, ele passa o token para seu par** usando a primitiva **`S-TOKEN-GIVE.request`**, como mostrado na Fig. 7-4(b).

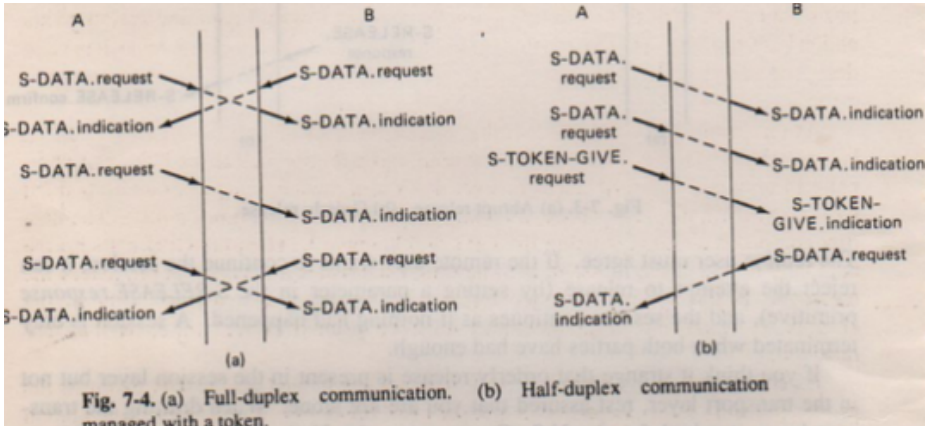


Fig. 7-4. (a) Full-duplex communication. (b) Half-duplex communication managed with a token.

7.1. 3 - DIALOG MANAGEMENT

- ... **O que acontece se o usuário que não está com o token quiser transmitir alguns dados?**
- ... Ele pode solicitar educadamente o token usando a primitiva `**S-TOKEN-PLEASE.request**`
- ... O detentor do token pode **concordar e passar o token, ou pode recusar, caso em que o outro usuário terá que esperar** (ou enviar uma mensagem de emergência usando dados acelerados, que não requerem o token).
- ...Se a operação full duplex for selecionada quando a sessão for estabelecida, **nenhum token é usado para transmissão de dados.**

7.1.4 - Sincronização

- **“Sincronização”..** Serviço usado para que, em caso de falha ou erro, as entidades possam **retornar a um estado conhecido**.
  - Apesar da camada de transporte oferecer recuperação de erros, ela não lida com falhas das camadas superiores.
-
-
- Exemplo prático .. Teletex
  - O dispositivo envia uma mensagem que será impressa em tempo real no destino.
  - Caso ocorra uma falha na impressora, o conteúdo pode ser perdido.
  - Nesse caso, a camada de transporte cumpriu seu papel perfeitamente, a falha ocorreu após a entrega dos dados.

7.1.4 - Sincronização

- **Solução ..** São inseridos pontos de sincronização durante a transmissão de dados.
  - **Ressincronização** .. Em caso de falha é possível voltar a um ponto anterior.
  - .. para isso ser possível, **o remetente deve manter os dados** enquanto necessários para uma possível recuperação.
  - Cada ponto de sincronização possui **número de série**.
  - Inserção de pontos e a ressincronização acontece por **primitivas**.
  - A camada de sessão apenas fornecer uma maneira de transmitir sinais numerados de sincronização e ressincronização.

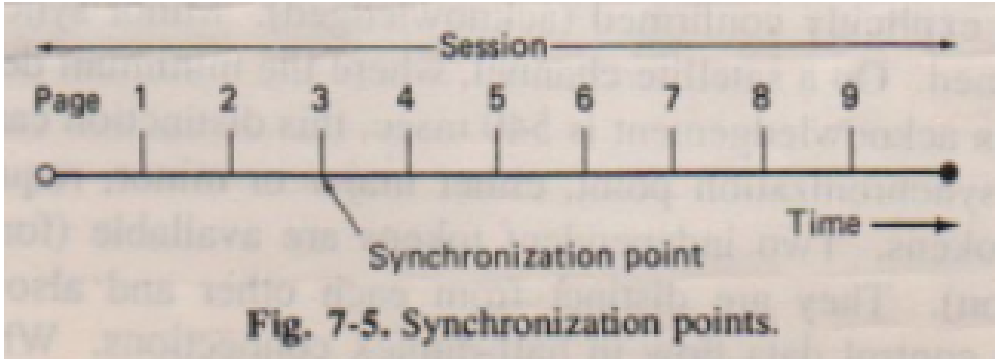


Fig. 7-5. Synchronization points.

7.1.4 - Sincronização

- Existem 2 tipos de pontos de sincronização:
  - “**Major**”.. Marcam blocos logicamente importantes(**unidades de diálogo**).
  - “**Minor**”.. Marcam progresso dentro de unidades de diálogo.

- A ressincronização só pode voltar até o **ponto maior mais recente**.
  - .. dados enviados até antes de um ponto de sincronização maior podem ser descartados com segurança.

- A escolha de qual ponto e onde utilizar cabe ao usuário.

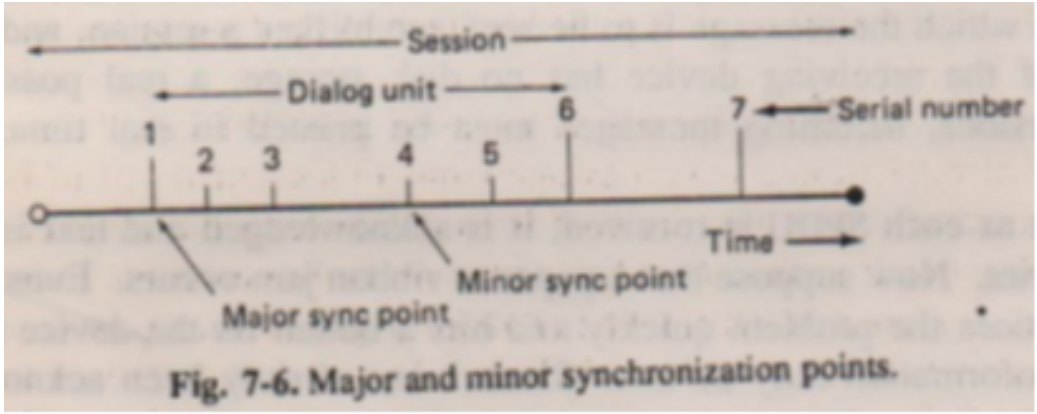


Fig. 7-6. Major and minor synchronization points.

7.1.4 - Sincronização

- .. Pontos maiores devem ser explicitamente confirmados(**acknowledge**).
  - .. pontos menores não são confirmados.

- **Diferença significativa**.. em alguns casos onde existe grande atraso para enviar uma mensagem e receber uma confirmação.
  - .. em satélites o atraso mínimo é de 540 ms.

- Definir pontos requer **tokens** específicos.
  - .. quando ocorre um ressincronização, tokens são restaurados

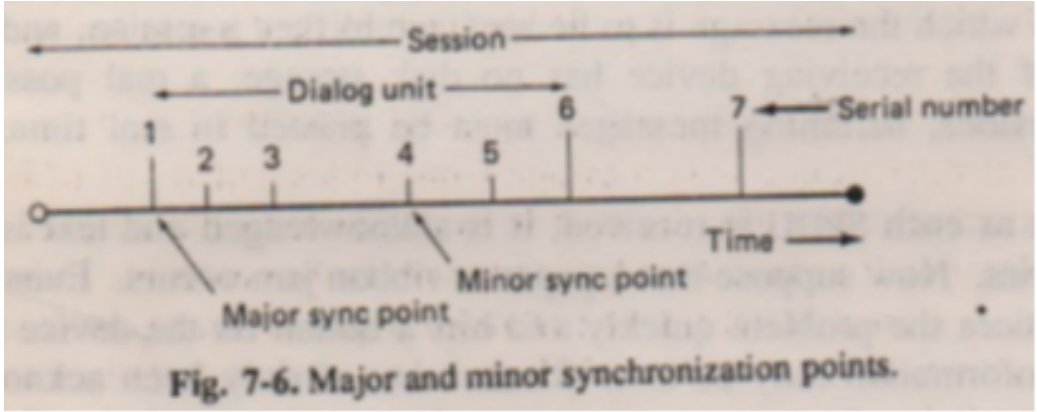


Fig. 7-6. Major and minor synchronization points.

7.1.5 - Gerenciamento de atividades

- “**Gerenciamento de atividades**”.. Recuso da camada de sessão que permite dividir o fluxo de mensagens em unidades lógicas, chamadas atividades.
  - .. **cada atividade é independente** de outras atividades.

- O usuário decide o que constitui uma atividade por meio de primitivas:
  - **S-ACTIVITY-START**.. para iniciar uma atividade.
  - **S-ACTIVITY-END**.. para encerrar uma atividade.

- A camada de sessão apenas executa primitivas.
  - Uma atividade cobre o tráfego enviado em ambas as direções

7.1.5 - Gerenciamento de atividades

- .."**Exemplo**".. Uma sessão para transferir arquivos.
  - Cada arquivo é definido como uma atividade, marcando o início e fim do arquivo.
  - Antes de enviar um arquivo, o remetente emite **S-ACTIVITY-START**.
  - Após concluir a transferência, **S-ACTIVITY-END** é usada par marcar o fim do arquivo

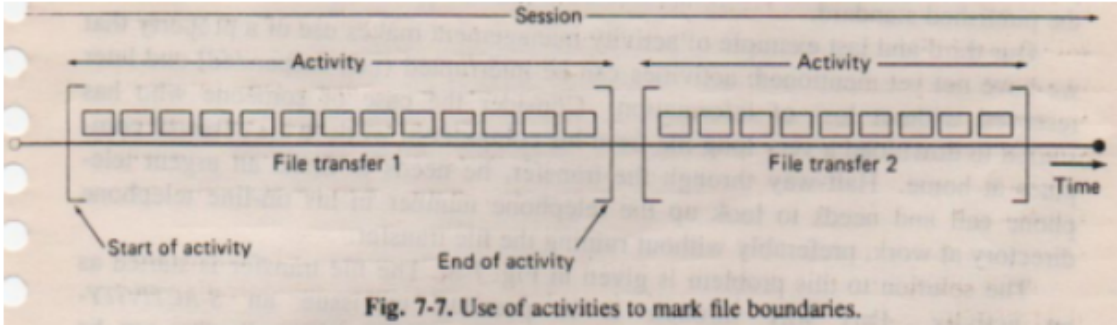


Fig. 7-7. Use of activities to mark file boundaries.

7.1.5 - Gerenciamento de atividades

- “**Quarentena**“.. mensagens são acumuladas antes de serem processadas.
  - O processamento começa após receber **S-ACTIVITY-END**.
  - Evita que falhas externas parem o sistema no meio de uma transação.

- “**Interrupções**“.. atividades podem ser interrompidas e retomadas sem perder informações.
  - “**S-ACTIVITY-INTERRUPT**”.. suspende a atividade.
  - “**S-ACTIVITY-RESUME**”.. retoma a atividade.

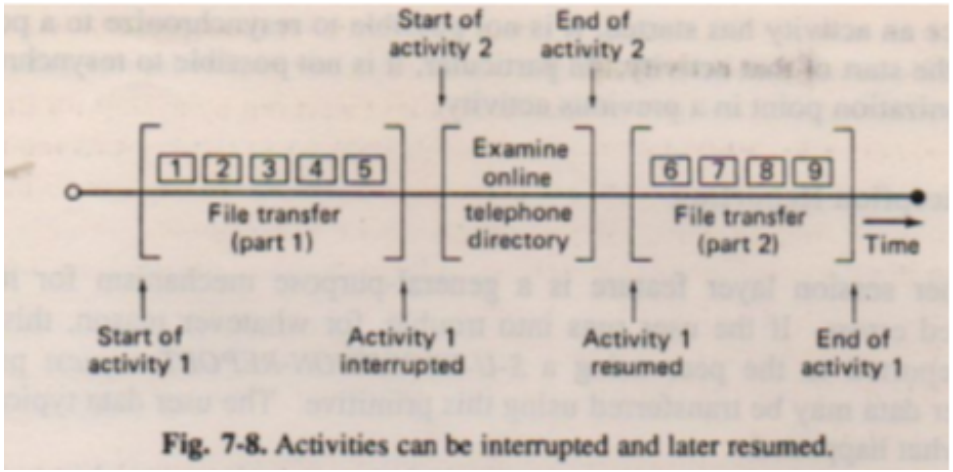


Fig. 7-8. Activities can be interrupted and later resumed.

7.1.5 - Gerenciamento de atividades

- “**Tokens**”.. Controlam o gerenciamento de atividades.
  - Para iniciar, interromper, retomar ou finalizar uma atividade é necessário um token de atividade.
  - **S-TOKEN-PLEASE** é a primitiva usada para solicitar token de outro usuário.

- Tokens podem ser passados ou solicitados
  - .. “**Problema**”.. caso um usuário tenha um token de atividade e outro o de sincronização menor, pode ocorrer um deadlock.
  - .. “**Solução**”.. para iniciar uma atividade ou sincronização o usuário deve possuir: token de atividade, token de sincronização menor e (se necessário) token de dados.
  - .. essa solução aumenta a complexidade de implementação.

7.1.5 - Gerenciamento de atividades

- Quando uma atividade é iniciada, um **ponto de sincronização maior é criado**.
  - .. é possível adicionar mais pontos de sincronização dentro da atividade.

- Não é possível ressincronizar para um ponto de sincronização de uma atividade anterior.
  - .. garantindo que cada atividade tenha um **escopo isolado de recuperação**.

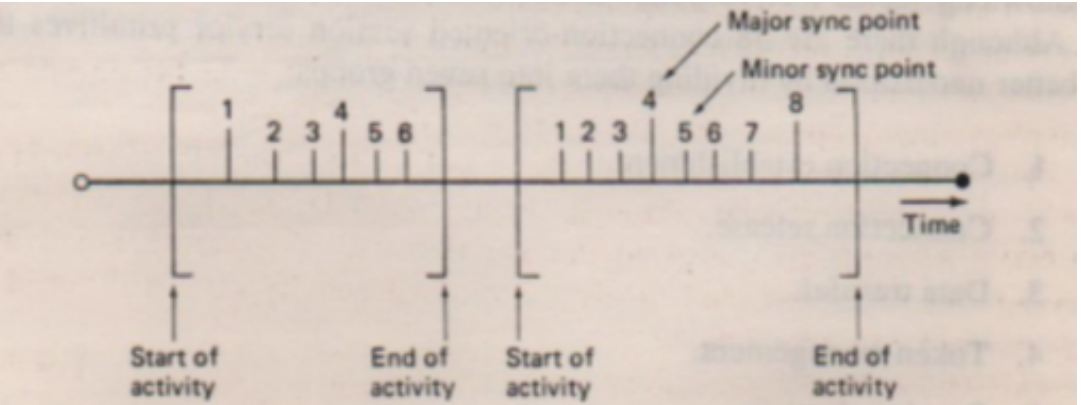


Fig. 7-9. Activities, major synchronization points, and minor synchronization points.

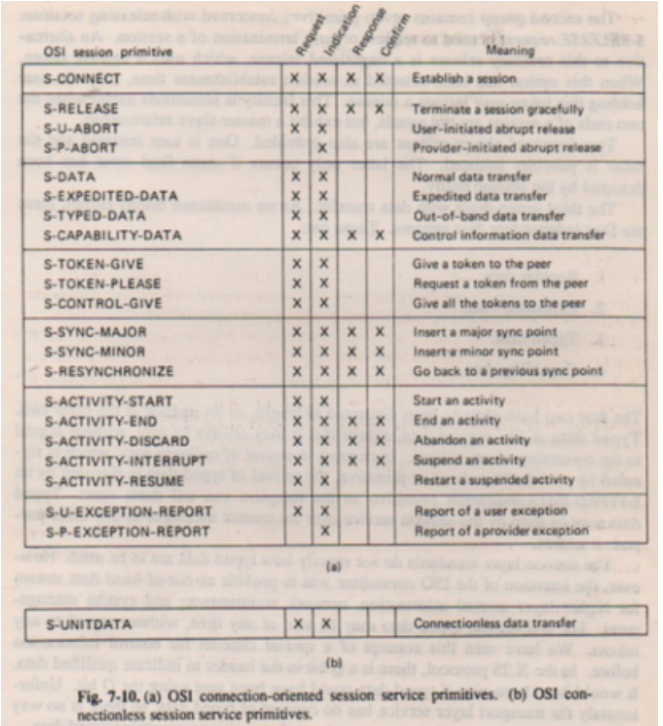
7.1.6 - Relatórios de Exceções

- “**Relatórios de Exceções**”.. mecanismo para relatar erros inesperados.
  - .. um erro pode ser relatado pelo usuário com a primitiva **S-U-EXCEPTION-REPORT.request**.
  - .. essa primitiva pode transferir dados do usuário, que informam o que ocorreu.

- .. o provedor da sessão também pode gerar uma exceção relacionada a problemas na camada de sessão ou em camadas inferiores.
  - .. a primitiva **S-P-EXCEPTION-REPORT.indication** é enviada ao usuário.
  - .. essa indicação descreve a natureza da exceção.
  - .. o usuário decide como lidar com a exceção.

7.1.7 - As Primitivas de Serviço de Sessão OSI

- "**Primitivas da camada de sessão**".. são 58 primitivas orientadas à conexão, organizadas em **7 grupos**.
  - Podem ser **request, indication, response e confirm**.
  - São usadas para estabelecer, gerenciar e encerrar sessões.
- Nem todas as combinações são válidas.
- Confirmações são tratadas nas camadas inferiores



OSI session primitive	Request	Indication	Response	Confirm	Meaning
S-CONNECT	X	X	X	X	Establish a session
S-RELEASE	X	X	X	X	Terminate a session gracefully
S-U-ABORT	X	X			User-initiated abrupt release
S-P-ABORT		X			Provider-initiated abrupt release
S-DATA	X	X			Normal data transfer
S-EXPEDITED-DATA	X	X			Expedited data transfer
S-TYPED-DATA	X	X			Out-of-band data transfer
S-CAPABILITY-DATA	X	X	X	X	Control information data transfer
S-TOKEN-GIVE	X	X			Give a token to the peer
S-TOKEN-PLEASE	X	X			Request a token from the peer
S-CONTROL-GIVE	X	X			Give all the tokens to the peer
S-SYNC-MAJOR	X	X	X	X	Insert a major sync point
S-SYNC-MINOR	X	X	X	X	Insert a minor sync point
S-RESYNCHRONIZE	X	X	X	X	Go back to a previous sync point
S-ACTIVITY-START	X	X			Start an activity
S-ACTIVITY-END	X	X	X	X	End an activity
S-ACTIVITY-DISCARD	X	X	X	X	Abandon an activity
S-ACTIVITY-INTERRUPT	X	X	X	X	Suspend an activity
S-ACTIVITY-RESUME	X	X			Restart a suspended activity
S-U-EXCEPTION-REPORT	X	X			Report of a user exception
S-P-EXCEPTION-REPORT		X			Report of a provider exception

(a)

S-UNITDATA	X	X			Connectionless data transfer

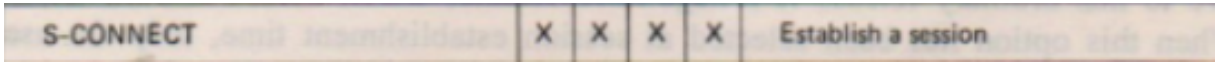
(b)

Fig. 7-10. (a) OSI connection-oriented session service primitives. (b) OSI connectionless session service primitives.

7.1.7 - As Primitivas de Serviço de Sessão OSI

- **Primeiro grupo**.. primitivas responsáveis por estabelecer a conexão.
  - S-CONNECT.request, S-CONNECT.indication, S-CONNECT.response e S-CONNECT.confirm.

- "**Parâmetros**".. identificador da sessão, endereços SSAP, qualidade de serviço, número inicial de ponto de sincronização, atribuição inicial dos tokens, dados do usuário(opcional), ativação de serviços(sincronização, gerenciamento de atividades e relatórios de exceção).



S-CONNECT	X	X	X	X	Establish a session

7.1.7 - As Primitivas de Serviço de Sessão OSI

- “**Segundo grupo**”.. liberação de sessões.
  - **S-REALEASE**.. encerra a sessão de forma ordenada.

  - **Liberação negociada**.. exige token de liberação
  - .. útil em uma relação mestre-escravo

  - **Liberação abrupta**
  - .. **S-U-ABORT**.. inicializada pelo usuário
  - .. **S-P-ABORT**.. inicializada pelo provedor

S-RELEASE	X	X	X	X	Terminate a session gracefully
S-U-ABORT	X	X			User-initiated abrupt release
S-P-ABORT		X			Provider-initiated abrupt release

7.1.7 - As Primitivas de Serviço de Sessão OSI

- “**Terceiro grupo**”.. Transferência de dados.
  - **S-DATA**.. dados comuns.
  - **S-EXPEDITED-DATA**.. dados urgentes, sem controle de token.
  - **S-TYPED-DATA**.. dados com tipo específico, podem ser enviados fora de banda, sem token.
  - **S-CAPACITY-DATA**.. usados para alterar parâmetros da sessão durante a sessão.
    - ..são confirmados e só podem ser enviados fora de atividades e em posse de tokens.

S-DATA	X	X			Normal data transfer
S-EXPEDITED-DATA	X	X			Expedited data transfer
S-TYPED-DATA	X	X			Out-of-band data transfer
S-CAPABILITY-DATA	X	X	X	X	Control information data transfer

7.1.7 - As Primitivas de Serviço de Sessão OSI

- “**Quarto grupo**”.. Gerenciamento de tokens.
  - .. A camada de sessão tem **quatro tipos tokens**.
    - .. tokens de dados, tokens de liberação, tokens de sincronização menor e tokens de atividade.
  - .. **S-TOKEN-GIVE**.. passa tokens para o par.
  - .. **S-TOKEN-PLEASE**.. solicita tokens.
  - .. **S-CONTROLE-GIVE**.. entrega todos os tokens, só pode ser usado fora de atividades.

S-DATA	X	X			Normal data transfer
S-EXPEDITED-DATA	X	X			Expedited data transfer
S-TYPED-DATA	X	X			Out-of-band data transfer
S-CAPABILITY-DATA	X	X	X	X	Control information data transfer

Token type	Controls
Data token	Data transfer in half-duplex mode
Release token	Initiation of orderly release
Synchronize-minor token	Insertion of minor sync points
Major/activity token	Activity or major sync operations

Fig. 7-11. The session layer tokens.

7.1.7 - As Primitivas de Serviço de Sessão OSI

- “**Quinto grupo**”.. Sincronização
  - Primitivas para **sincronização, maior, menor e ressincronização**.
  - .. todas são confirmadas.
  - .. usam números de série(0-999.999) para identificar pontos.
  - .. exigem tokens de sincronização.

S-SYNC-MAJOR	X	X	X	X	Insert a major sync point
S-SYNC-MINOR	X	X	X	X	Insert a minor sync point
S-RESYNCHRONIZE	X	X	X	X	Go back to a previous sync point

7.1.7 - As Primitivas de Serviço de Sessão OSI

- “**Sexto grupo**”.. Gerenciamento de atividades
  - .. **iniciam, interrompem, retomam, finalizam ou descartam** atividades.
  - .. todas as operações exigem tokens.

S-ACTIVITY-START	X	X			Start an activity
S-ACTIVITY-END	X	X	X	X	End an activity
S-ACTIVITY-DISCARD	X	X	X	X	Abandon an activity
S-ACTIVITY-INTERRUPT	X	X	X	X	Suspend an activity
S-ACTIVITY-RESUME	X	X			Restart a suspended activity

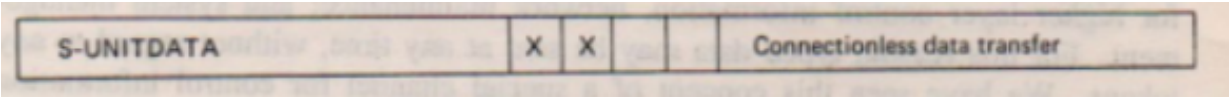
7.1.7 - As Primitivas de Serviço de Sessão OSI

- “**Sétimo grupo**“.. Relatório de exceções.
  - .. **S-P-EXCEPTION-REPORT**.. emitida pelo provedor.
  - .. **S-U-EXCEPTION-REPORT**.. emitida pelo usuário

S-U-EXCEPTION-REPORT	X	X			Report of a user exception
S-P-EXCEPTION-REPORT		X			Report of a provider exception

7.1.7 - As Primitivas de Serviço de Sessão OSI

- “**S-UNITDATA**”.. Primitiva para comunicação sem conexão.
  - .. útil para mensagens pontuais ou transmissões simples
  - .. **não é confiável**, pois não há controle de sessão ou mecanismo de recuperação.



S-UNITDATA	X	X			Connectionless data transfer

7.2.Chamada de procedimento remoto

Ponto de Partida: Papel da Camada de Sessão (gerenciamento de diálogo e erros).

A Alternativa: RPC como um novo modelo radical, sem conexão.

Ampla Utilização: Implementado em redes e sistemas distribuídos.

Conflito com o OSI: Não se encaixa bem no modelo tradicional.

Foco Principal: Projetado para ser rápido, sem estrutura multicamadas.

Posicionamento Flexível: Lógica de camada de sessão , mas implementável na camada de aplicação.

7.2.1. O Modelo Cliente-Servidor

Ponto de Partida: Quebra do modelo de comunicação simétrico.

Definição dos Papéis: Clientes: Solicitam os recursos (ex: estações de trabalho sem disco).

Servidor: Provê os recursos e realiza o trabalho (ex: servidor de arquivos com disco).

Fluxo da Comunicação: Pares Requisição-Resposta. Iniciativa sempre do cliente.

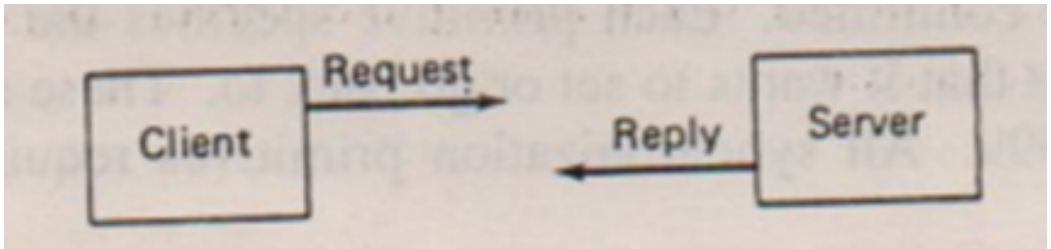


Fig. 7-12. No modelo cliente-servidor, o cliente envia uma requisição e o servidor envia uma resposta.

Por que o RPC? As Limitações do Modelo Tradicional

Problema 1: Sobrecarga (Overhead) Sessões com conexão têm alta sobrecarga, o que é pouco atraente para aplicações onde o desempenho é crítico (ex: servidores de arquivos). Uma comunicação sem conexão sobre datagramas é, muitas vezes, uma escolha melhor.

Problema 2: A Base Conceitual Mesmo sem conexão, a base conceitual da comunicação continua sendo E/S (Entrada/Saída). Programas se comunicam com comandos de baixo nível (ex: X-DATA.request) e interrupções.

Conclusão Essas não são as ferramentas adequadas para construir aplicações bem estruturadas.

A Grande Ideia do RPC: De I/O para Chamadas de Procedimento

Uma Perspectiva Diferente: A corrente de pensamento do RPC aborda o modelo cliente-servidor de uma perspectiva completamente diferente.

A Analogia Central: Enviar uma Mensagem e obter uma Resposta é como um programa que Chama um Procedimento e obtém um Resultado.

Comportamento Síncrono: Em ambos os casos, o chamador (caller) inicia uma ação e espera (fica bloqueado) até que ela seja concluída e os resultados estejam disponíveis.

Ocultando a Localização: O chamador não precisa estar ciente se o procedimento está executando na mesma máquina ou em uma máquina diferente e remota.

O Papel dos Stubs

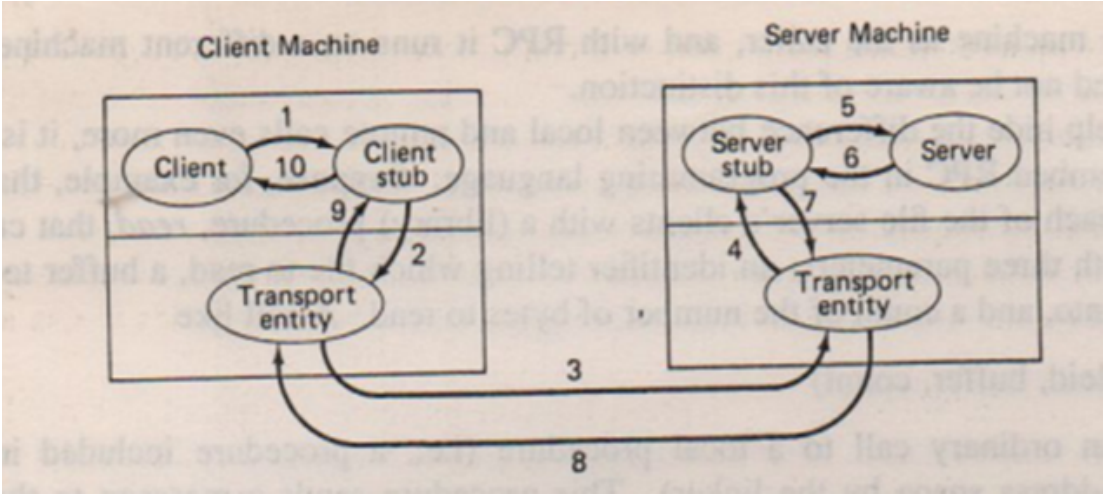
O Mecanismo: Para ocultar a complexidade da rede, usamos procedimentos locais chamados Stubs. O programador chama o stub como se fosse uma função local comum.

O que o Stub Faz? O stub envia a mensagem ao servidor de arquivos. Aguarda (bloqueado) pela resposta. Somente após a chegada da resposta, o stub devolve o controle e o resultado ao chamador.

O Objetivo Final: A comunicação assume a forma de chamadas de procedimento, não comandos de E/S. A meta é fazer com que uma chamada remota não pareça diferente de uma chamada local.

07 - Session Layer /7.2.2 - Implementação do RPC

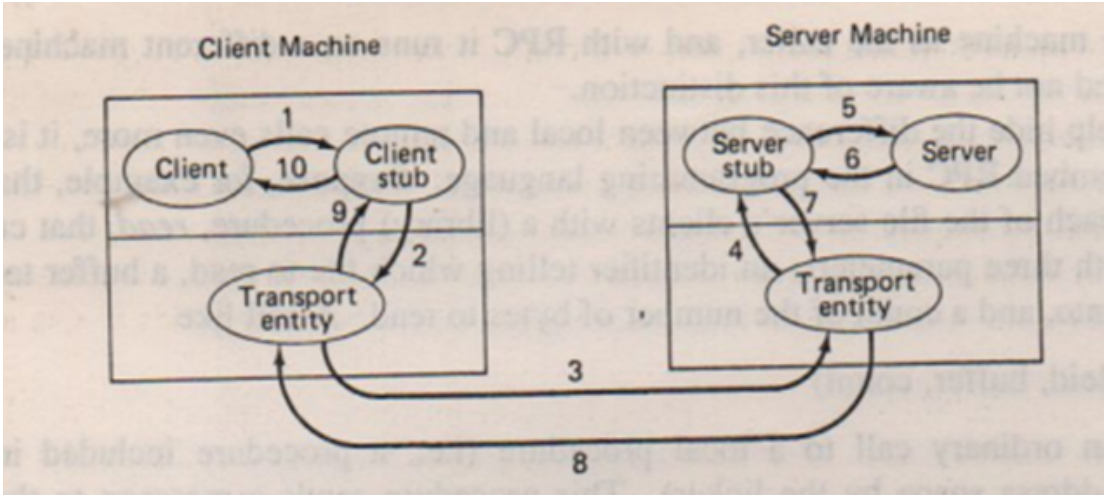
Implementação do RPC: Lado do Cliente



Passo 1: A Chamada Local O cliente chama o procedimento stub localmente, como se fosse qualquer outra função. Para o programa cliente, a chamada é normal e local, nada parece incomum. Marshalling (Empacotamento) O stub do cliente coleta os parâmetros e os empacota em uma mensagem. Essa operação é conhecida como empacotamento de parâmetros (parameter marshalling).

Passo 2 e 3: Envio A mensagem é entregue à camada de transporte (Passo 2) e colocada na rede (Passo 3). Em muitos sistemas, o Passo 2 é uma trap para o sistema operacional.

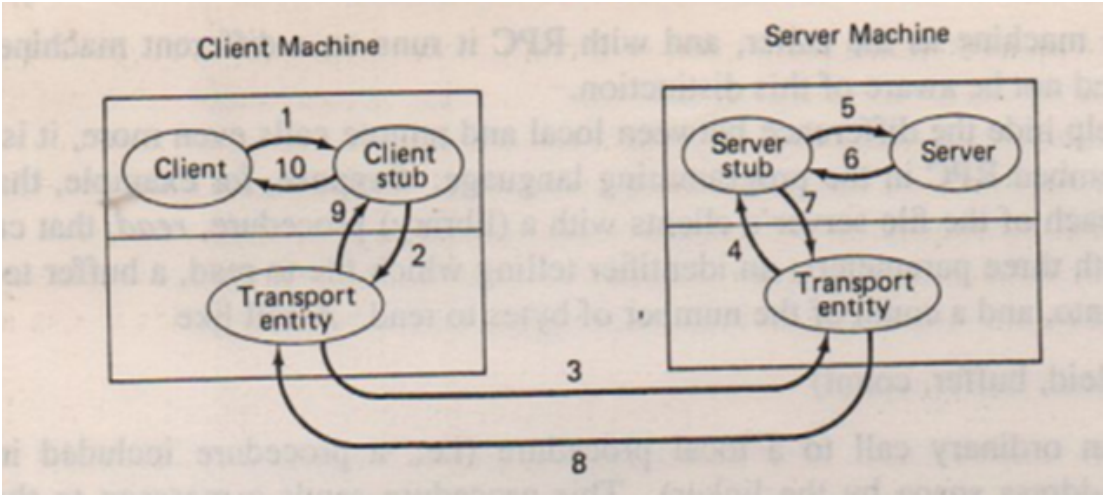
Implementação do RPC: Lado do Servidor (Passos 4-7)



Passo 4: Chegada e Unmarshalling: A mensagem chega e a entidade de transporte a entrega ao stub do servidor. O stub desempacota os parâmetros (a operação inversa do marshalling).

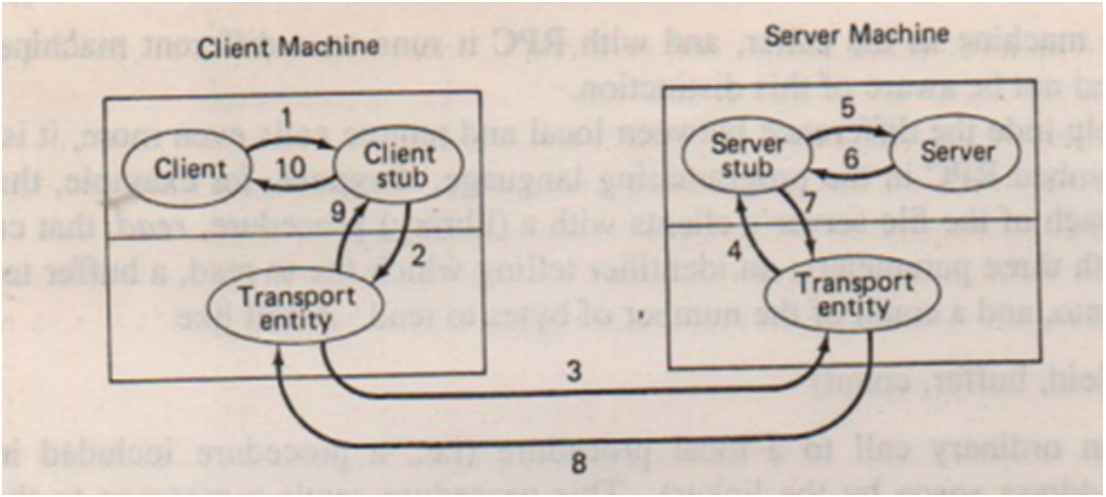
Passo 5: A Chamada no Servidor O stub do servidor chama o procedimento real do servidor, passando os parâmetros. O procedimento do servidor não tem como saber que foi ativado remotamente. Apenas os stubs sabem da comunicação remota.

Implementação do RPC: Lado do Servidor (Passos 4-7)



Passo 6 e 7: O Retorno O servidor termina seu trabalho e retorna o resultado ao seu stub (Passo 6). O stub do servidor empacota o resultado em uma nova mensagem e a entrega à camada de transporte (Passo 7).

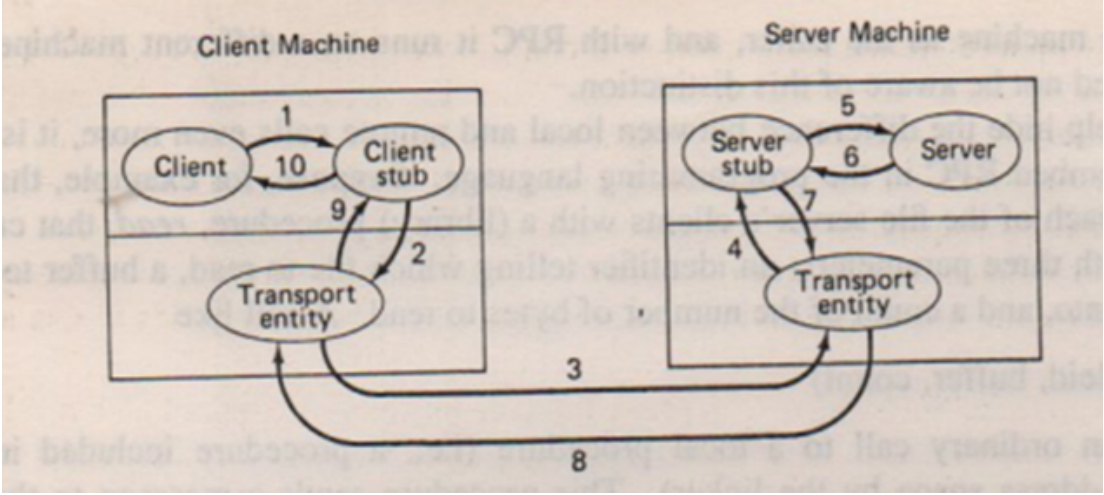
Implementação do RPC: O Retorno e a Transparência (Passos 8-10)



Passo 8 e 9: Retorno ao Cliente: A resposta viaja pela rede (Passo 8) e é entregue pela entidade de transporte ao stub do cliente (Passo 9).

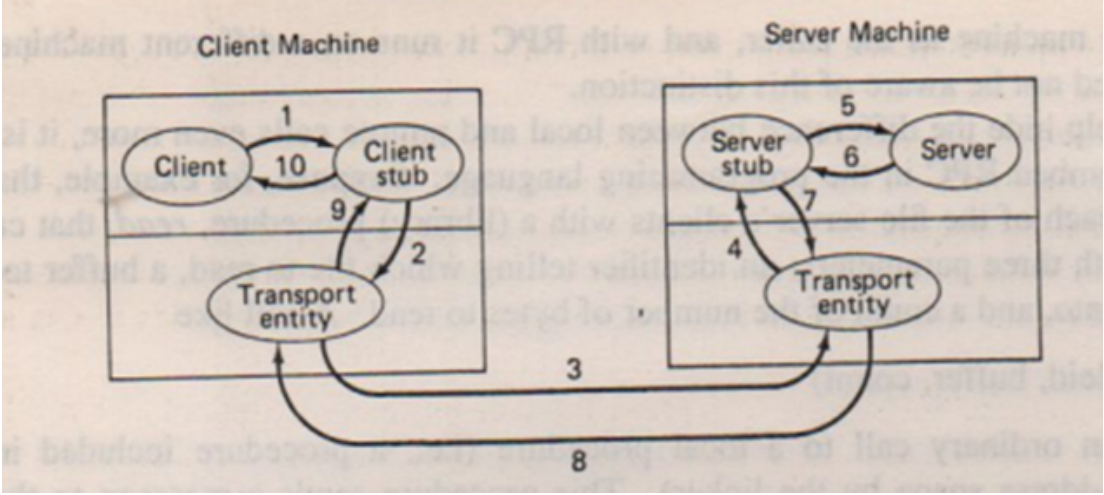
Passo 10: Entrega Final O stub do cliente desempacota o resultado e o retorna ao programa cliente original, finalizando a chamada.

Implementação do RPC: O Retorno e a Transparência (Passos 8-10)



O Propósito Final: Transparência: Todo este mecanismo de 10 passos serve para criar a ilusão de uma chamada direta ao procedimento do servidor distante. Quando a ilusão funciona, diz-se que o mecanismo é transparente. No entanto, alcançar a transparência total tem suas dificuldades.

Implementação do RPC: O Retorno e a Transparência (Passos 8-10)



O Propósito Final: Transparência: Todo este mecanismo de 10 passos serve para criar a ilusão de uma chamada direta ao procedimento do servidor distante. Quando a ilusão funciona, diz-se que o mecanismo é transparente. No entanto, alcançar a transparência total tem suas dificuldades.

Dificuldades com a Transparência: Passagem por Referência

O Principal Problema: A passagem de parâmetros.

O que é fácil: Passagem por Valor Inteiros, strings, arrays, etc., são simplesmente colocados na mensagem pelo stub.

O Desafio: Passagem por Referência (Ponteiros) Localmente: Um ponteiro (endereço de memória) é passado diretamente e funciona perfeitamente. Remotamente: A mesma estratégia falha completamente.

Por que Falha? O endereço de memória do cliente é inválido e não faz sentido na máquina do servidor. Resultado: O servidor tenta usar o ponteiro, obtém o valor errado e a computação falha

Dificuldades com a Transparência: Passagem por Referência

O Principal Problema: A passagem de parâmetros.

O que é fácil: Passagem por Valor Inteiros, strings, arrays, etc., são simplesmente colocados na mensagem pelo stub.

O Desafio: Passagem por Referência (Ponteiros) Localmente: Um ponteiro (endereço de memória) é passado diretamente e funciona perfeitamente. Remotamente: A mesma estratégia falha completamente.

Por que Falha? O endereço de memória do cliente é inválido e não faz sentido na máquina do servidor. Resultado: O servidor tenta usar o ponteiro, obtém o valor errado e a computação falha

Uma Solução Possível: Cópia/Restauração (Copy/Restore)

A Alternativa: Substituir a chamada por referência pelo mecanismo de chamada por cópia/restauração (copy/restore).

O Processo em 3 Etapas:

1. CÓPIA (Cliente → Servidor): O stub do cliente localiza o item apontado e envia uma cópia do seu valor para o servidor.

2. USO LOCAL (No Servidor): O stub do servidor armazena essa cópia em sua memória e passa um novo ponteiro local para o procedimento do servidor, que o utiliza normalmente.

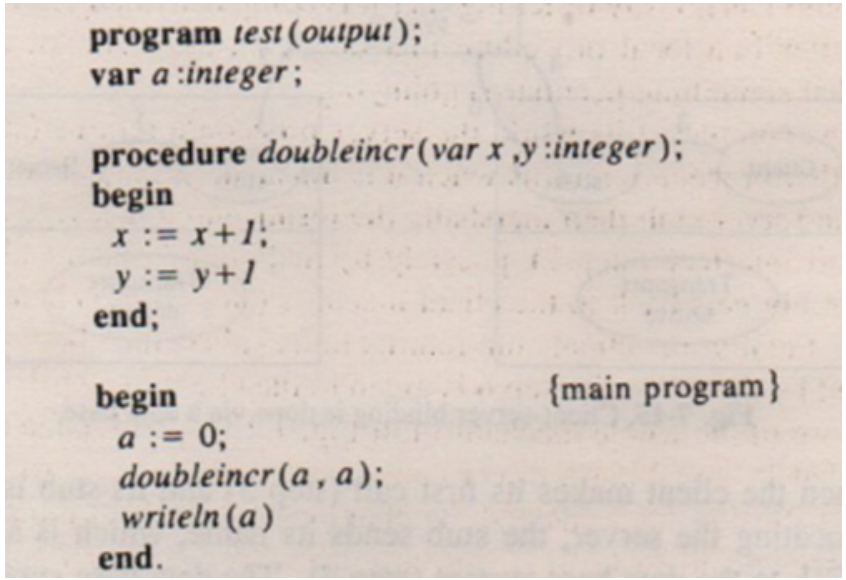
3. RESTAURAÇÃO (Servidor → Cliente): Após a execução, o stub do servidor envia o item (possivelmente modificado) de volta. O stub do cliente usa o valor recebido para sobrescrever o original

07 - Session Layer / 7.2.2 - Implementação do RPC

O mecanismo de copy/restore pode falhar em certas situações patológicas.

Execução Local (Resultado Correto): Na chamada doubleincr(a, a), ambos os parâmetros são ponteiros para a mesma variável a. A variável a é incrementada duas vezes, resultando no valor 2.

Execução Remota (Resultado Incorreto): O stub do cliente envia duas cópias independentes de a para o servidor. O servidor incrementa cada cópia para 1 e envia ambas de volta. O stub do cliente restaura a para 1, e depois o restaura para 1 novamente. Resultado final incorreto:



```
program test(output);
var a:integer;

procedure doubleincr(var x,y:integer);
begin
  x := x+1;
  y := y+1
end;

begin                                {main program}
  a := 0;
  doubleincr(a, a);
  writeln(a)
end.
```

Implicações e a Quebra da Transparência

Problemas Adicionais: Ponteiros em meio a estruturas de dados complexas (listas, grafos) também são muito problemáticos. Parâmetros que são, eles mesmos, procedimentos ou funções também são difíceis de lidar.

A Solução Comum (e Imperfeita): Muitos sistemas RPC simplesmente proíbem o uso de ponteiros, referências e funções como parâmetros em chamadas remotas.

A Consequência: Quebra da Transparência: Essa decisão facilita a implementação do sistema RPC. Mas, ela quebra o objetivo da transparência, porque as regras para chamadas locais e remotas se tornam diferentes.

A Solução: Registro e Binding Dinâmico

A Questão: Como o cliente encontra o endereço do servidor em um ambiente dinâmico? A Solução (Birrell e Nelson): Usar um sistema de banco de dados especializado.

Lado do Servidor: Registro (Passos 1-2) Ao inicializar, o servidor se registra no banco de dados. Envia: Nome, Endereço de Rede e um Identificador Único.

Lado do Cliente: Binding (Passos 3-5) Na primeira chamada, o stub do cliente consulta o BD com o nome do servidor. O BD retorna o endereço e o ID único do servidor. Este processo é chamado de vinculação (binding) e só precisa ser feito uma vez.

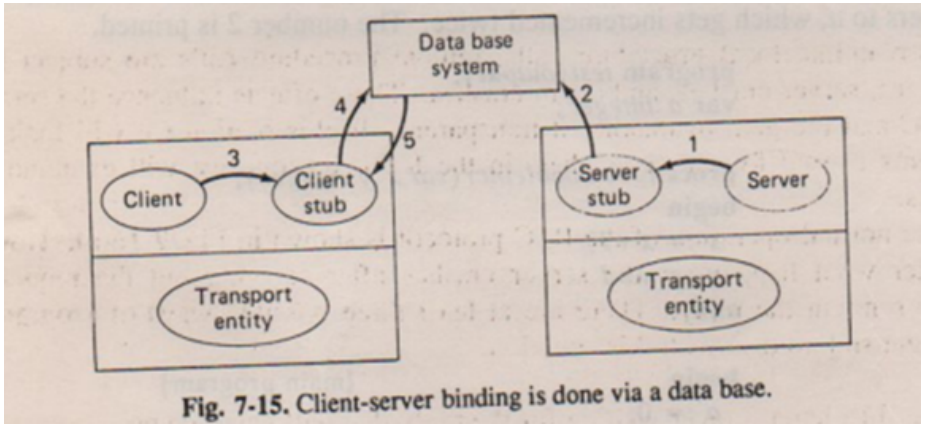


Fig. 7-15. Client-server binding is done via a data base.

O Papel do Identificador Único

O identificador único de 32 bits é incluído em cada chamada RPC.

Função 1: Roteamento de Mensagens: É usado pela entidade de transporte do servidor para saber a qual dos muitos stubs de servidor possíveis a mensagem de entrada deve ser entregue.

Função 2: Detecção de Falhas (Crashes) Se o servidor falha e reinicia, ele se registra com um NOVO número único. Tentativas do cliente de usar o ID antigo irão falhar. Essa falha força o cliente a refazer a vinculação (rebind), tornando-o ciente da falha do servidor.

O Protocolo RPC: Confiabilidade e Duplicatas

Estrutura Simples: O protocolo pode consistir em apenas duas mensagens: Requisição e Resposta.

Conteúdo das Mensagens: Identificador Único do Servidor. Identificador de Transação. Parâmetros.

Mecanismo de Confiabilidade: Retransmissão: O stub do cliente pode definir um temporizador (timer) ao enviar uma requisição. Se o timer expirar sem resposta, a requisição pode ser retransmitida.

Lidando com Duplicatas: O Identificador de Transação permite ao servidor reconhecer e rejeitar requisições duplicadas. Isso exige que o servidor mantenha um registro do último ID de transação recebido de cada cliente

A Realidade do RPC: Tratamento de Exceções

RPC vs. Chamada Local: Muitas coisas podem dar errado em uma chamada remota (ex: o servidor pode estar fora do ar).

A Solução: Lançar uma Exceção: Em caso de erro, o controle não deve simplesmente retornar ao chamador. O sistema deve lançar uma exceção para ser tratada por um manipulador de exceções (exception handler). Dependência da Linguagem: O projeto do mecanismo de tratamento de exceções depende da linguagem de programação utilizada. Objetivo Final: É necessário um método claro para distinguir chamadas bem-sucedidas das mal-sucedidas

7.2.3. Semântica da Chamada de Procedimento Remoto

RPC vs. Chamada Local: RPC está sujeito a perda de mensagens e falhas de cliente/servidor. Isso influencia a semântica e o objetivo da transparência.

Cenário Crítico: O servidor executa a requisição, mas falha ANTES de enviar a resposta.

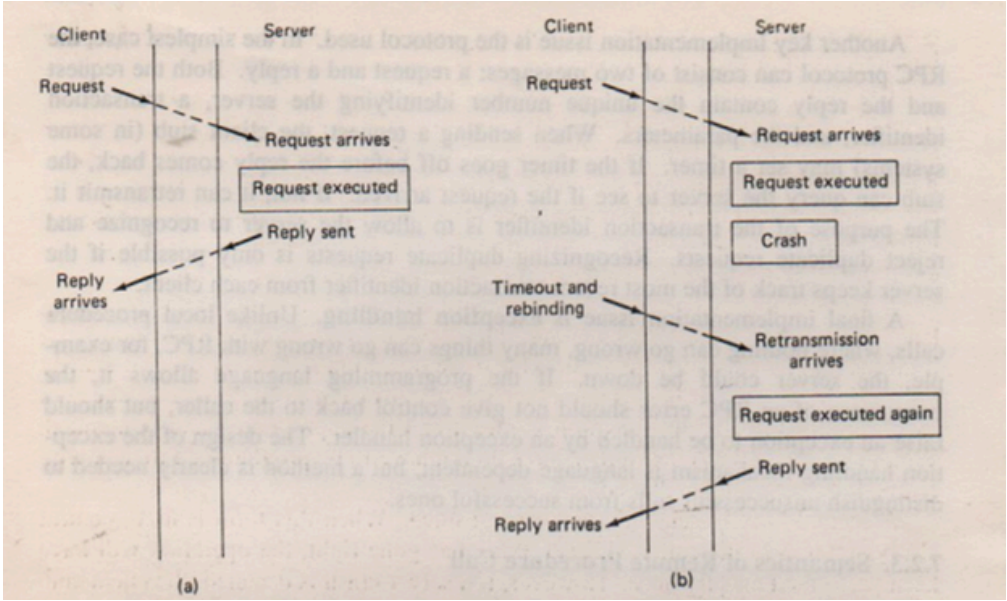
O que o cliente faz? 3 Abordagens Possíveis: Aguardar indefinidamente pela resposta que nunca virá. Expirar o tempo (timeout) e lançar uma exceção ou relatar falha. Expirar o tempo e retransmitir a requisição

A Consequência da Retransmissão: Idempotência

Risco da Retransmissão: A abordagem de retransmitir pode fazer com que a operação seja executada duas vezes (veja a figura b). Isso pode acontecer mesmo sem falhas, se apenas a resposta for perdida na rede.

Conceito-Chave: Idempotência Operações que podem ser repetidas várias vezes sem causar problemas são chamadas de idempotentes.

Exemplos: Idempotente: Ler um bloco de um arquivo. Não Idempotente: Anexar dados a um arquivo ou transferir dinheiro.



A Semântica Ideal (e Inatingível)

O Objetivo: A semântica mais desejável é exatamente uma vez (exactly once).
Garantia: Cada chamada é executada exatamente uma vez, nem mais, nem menos.

A Realidade: Esse objetivo é inatingível. Após uma falha de servidor, é impossível saber se a operação foi realizada ou não antes do crash. (Ex: a fábrica de chocolate).

Uma Versão Possível (com Exceções): Retorno Normal: A operação foi executada exatamente uma vez.

Erro / Crash: A chamada não retorna, mas lança uma exceção. A responsabilidade de lidar com o erro passa para o manipulador de exceções (exception handler)

As Semânticas Realistas: Fazendo uma Escolha

Opção 1: No Máximo Uma Vez (At Most Once) O stub não retransmite; em caso de falha, desiste e retorna um erro.
Garantia: A operação foi executada 0 ou 1 vez. Ideal para operações sensíveis, onde a duplicação é inaceitável. A recuperação fica a cargo do cliente.

Opção 2: Pelo Menos Uma Vez (At Least Once) O stub continua tentando (retransmitindo) até obter uma resposta.
Garantia: A operação foi executada 1 ou mais vezes. Ideal para operações idempotentes. Variação útil: Última de Muitas (Last of Many) garante que o resultado retornado é o da operação final.

Resumo: O Desafio da Transparência em RPC

O objetivo de uma semântica de RPC totalmente transparente é complicado pela possibilidade de falhas que afetam o servidor, mas não o cliente. Esse é um problema exclusivo de sistemas distribuídos. Em um sistema com um único processador, uma falha que elimina o servidor também elimina o cliente. Nesse caso, toda a computação é totalmente perdida, não deixando espaço para a ambiguidade sobre o estado da operação.

7.2.4 - Orphans

- Um órfão acontece quando um processo faz uma chamada de procedimento remoto (RPC) ou inicia alguma atividade em uma sessão, mas cai/trava antes de receber a resposta.

- É um problema pois:
  - consome recursos (ex:. cilcos de CPU)
  - pode negar acesso de arquivos e objetos para processos que tem o direito de acessa-los.
  - se o cliente fazer um reboot e começar um RPC denovo, os resultados enviados pelo órfão pode gerar confusão e inconsistência.

- Existe 4 maneiras de se lidar com órfão:
  - Extermínio: matar imediatamente qualquer computação órfã detectada.
  - Expiração: cada computação tem um prazo de validade; se passar do tempo, encerra automaticamente.
  - Reencarnação: quando um processo reinicia, ele cancela qualquer tarefa inacabada da “vida anterior”.
  - Reencarnação suave: em vez de cancelar à força, o novo processo pergunta se existem tarefas ligadas à sua versão anterior e as encerra.

7.2.4 - Orphans

- Extermínio:
  - Necessita de lista de RPCs pendentes (registro resiliente a falhas do processador) para quando a máquina falhar percorrer a lista e ir deletando recursivamente as RPCs pendentes (órfãos podem gerar filhos, netos, etc por isso remover recursivamente)
- Expiração: cada computação tem um prazo de validade; se passar do tempo, encerra automaticamente.
- Reencarnação: quando um processo reinicia, ele cancela qualquer tarefa inacabada da “vida anterior”.
- Reencarnação suave: em vez de cancelar à força, o novo processo pergunta se existem tarefas ligadas à sua versão anterior e as encerra.

Referências Bibliográficas

- Andrew S. Tanenbaum - “Computer Networks” - Prentice Hall; Englewook Cliffs; New Jersey; 1989; 2nd Edition.